

# **CP372 – Assignment 01**

# RFC: Bulletin Board Client–Server Protocol

**Document Status:** Informational

**Intended Audience:** CP372 Assignment 01

**Authors:** Abdul Aziz Hyder, Syed Sabilullah Hussaini

**Version:** 1.0

**Date:** 22 January 2026

## 1. Introduction

### 1.1 Purpose of This Document

The CP372 Bulletin Board Client-Server System's application-layer protocol is described in this RFC. The message formats, command semantics, server responses, error codes, and concurrency guarantees that define client-server interaction should all be formally specified in this RFC. By acting as a contract between the client and server implementations, this RFC guarantees the accuracy, dependability, and interoperability of the distributed system.

### 1.2 Scope

This RFC covers:

The protocol for client-server communication

Semantics and syntax of messages

Error-handling actions

Does not cover :

GUI layout and visual design are not covered by this RFC. Long-term persistence (the board is not persistent) Encryption or other network security measures

### 1.3 Design Goals

The protocol is designed with the following goals:

- Simplicity and clarity
- Robust handling of malformed or invalid requests
- Concurrency safety under multiple clients
- Explicit and structured error reporting
- Ease of manual testing using text-based commands

## 2. System Overview

### 2.1 Architectural Model

The architecture of the system is traditional centralized client-server:

- The bulletin board is kept up to date by a single centralized server.
- Several clients could connect at once.
- To communicate, TCP stream sockets are used.
- Requests and responses are the basis of interaction.

Because all validation, synchronization, and state management are done server-side, the server is authoritative.

### 2.2 Connection lifecycle:

The following is how the connection lifecycle goes:

1. Command-line parameters specifying board dimensions, note dimensions, and acceptable colours are used to launch the server.
2. A TCP connection to the server is started by a client.
3. When a connection is made, the server transmits a configuration handshake message that includes:
4. Board height and width
5. Take note of the height and width.
6. List of acceptable hues
7. After that, the client can give commands until DISCONNECT is sent.

The server gracefully terminates the connection upon DISCONNECT.

### 3. TERMINOLOGY AND DEFINITIONS

- **Board:** The server-controlled rectangular bulletin board.
- **Note:** A rectangular note with a set width and height that is placed on the board.
- **Pin:** A marker inserted into a message at a certain coordinate.
- A note with at least one active pin is referred to as a pinned note.
- **Client:** A graphical user interface program that establishes a connection with the server and sends commands.
- **Server:** The centralized procedure that maintains board status and upholds all regulations.
- An atomic action is one that all customers perceive as being indivisible.
- Two notes that occupy the exact same rectangular area are said to completely overlap.

## 4. Data Model

### 4.1 Coordinate System

- The upper-left corner of the board is where the origin (0,0) is situated.
- To the right, the x-axis rises.
- The y-axis descends.
- Every coordinate is an integer that is not negative.

### 4.2 Board Properties

- The board's height and width are set at server startup.
- A note must be entirely contained within the board's perimeter.
- Notes must not entirely overlap, although they may partially overlap.

### 4.3 Note Properties

Every note is characterized by:

- Coordinate on the upper left (x, y)
- fixed height and breadth
- colour selected from the list of acceptable colours on the server
- Content of the message (string)
- Pin status, obtained from related pins

A note is considered pinned if it has at least one active pin.

### 4.4 Pin Properties

A coordinate (x, y) defines a pin.

Only inside an already-existing note may a pin be inserted.

Each note may have more than one pin.

A pin can only be removed to unpin a note if it was the final pin left.

## 5. Protocol Overview

### 5.1 Transport Protocol

TCP is used for communication.

A byte stream is used to transfer messages.

Every message ends with a single newline character (`\n`) and is line-oriented.

### 5.2 Message Exchange Pattern

Every communication is started by the customer.

The server responds to every client request exactly once.

For each connection, the protocol is strictly synchronous.

## 6. Message Format Specification

### 6.1 General Message Syntax

Plain-text ASCII strings make up messages.

Whitespace is used to divide fields.

Depending on the command, parameters can be either fixed or key-value.

Every message has a newline character at the end.

### 6.2 Formal Grammar (Informal BNF)

`<command> ::= POST <note>`

    | GET <request>

    | GET PINS

    | PIN <x> <y>

    | UNPIN <x> <y>

    | SHAKE

    | CLEAR

    | DISCONNECT

<response> ::= OK <payload>  
              | ERROR <error\_code> <message>

## 7. Client-to-Server Commands

### 7.1 POST Command

**Syntax:**

POST <x> <y> <color> <message>

**Preconditions:** Non-negative coordinates are required.

The note needs to be completely inside the board.

Colour needs to be supported.

A previous note cannot be entirely overlapped by this one.

Success Response:

OK POST id=<id>

**Error Responses:**

- OUT\_OF\_BOUNDS
- COLOR\_NOT\_SUPPORTED
- COMPLETE\_OVERLAP

### 7.2 GET Command

**GET PINS**

**Syntax:**

GET PINS

**Response:**

Returns all pin coordinates.

**Filter-based GET**

**Syntax:**

GET [color=<color>] [contains=<x> <y>] [refersTo=<substring>]

Missing criteria imply ALL.

**Response:**

OK NOTES count=<n> notes=...

### 7.3 PIN Command

**Syntax:**

PIN <x> <y>

Semantics: Applies to every note that has a coordinate.

All of the notes are pinned if they overlap.

**Error:**

- NO\_NOTE\_AT\_COORDINATE

**7.4 UNPIN Command****Syntax:**

UNPIN <x> <y>

**Semantics:**

- Removes one pin at the coordinate from all applicable notes.

**Error:**

- PIN\_NOT\_FOUND

**7.5 SHAKE Command****Syntax:**

SHAKE

**Semantics:**

Removes all unpinned notes.  
Operation is atomic.

**Response:**

OK SHAKE removed=<n>

**7.6 CLEAR Command****Syntax:**

CLEAR

**Semantics:**

Removes all notes and all pins.  
Operation is atomic.

**7.7 DISCONNECT Command****Syntax:**

DISCONNECT

**Semantics:**

gracefully ends the connection.

**8. Server-to-Client Responses****8.1 Success Responses**

All successful responses begin with:  
OK <payload>

Examples:

- OK POST id=3
- OK NOTES count=2 notes=...
- OK PINS count=4 points=...

## 8.2 Error Responses

All errors use the form:

ERROR <ERROR\_CODE> <human-readable message>

# 9. Error Handling

## 9.1 Error Classification

Defined error codes include:

- INVALID\_FORMAT
- OUT\_OF\_BOUNDS
- COLOR\_NOT\_SUPPORTED
- COMPLETE\_OVERLAP
- NO\_NOTE\_AT\_COORDINATE
- PIN\_NOT\_FOUND

## 9.2 Client-Side Responsibilities

- Verify numerical input wherever you can.
- Avoid leaving necessary fields empty.
- Don't send orders that are incorrect.

## 9.3 Server-Side Responsibilities

- Find all requests that are incorrect or faulty
- Never let client errors cause a crash.
- Return explicit ERROR replies at all times.

# 10. Concurrency and Synchronization

## 10.1 Multithreading Model

- A thread-per-client paradigm is employed by the server.
- A specific ClientHandler thread manages each connection.

## 10.2 Shared Data Protection

- All board states are stored in a single shared BoardState object.
- Access is protected using a read-write lock.
- GET operations use read locks.
- POST, PIN, UNPIN, SHAKE, and CLEAR use write locks.

## 10.3 Consistency Guarantees

Customers see a steady state of the board.

Atomic operations guarantee the absence of intermediate states.

## 11. Border and Failure Cases

The system handles:

- At starting, the board is empty.
- GET yields no results.
- The server is operating without any customers.
- Mid-session client disconnecting
- Graceful handling of unexpected disconnects

## 12. Security Considerations

- Every input is server-side verified.
- The system is predicated on a reliable academic setting.
- It is not relevant to discuss formal cryptographic security.

## 13. Implementation Notes (Non-Normative)

- ServerSocket is used to implement the server as a Java console program.
- Java Swing is used to implement the client.
- The board is non-persistent and exists only during server runtime.

## 14. Division of Responsibilities

- Student Name 1: Mainly in charge of implementing the server  
Student Name 2: Mainly in charge of implementing the client GUI
- Protocol design, testing, drafting RFCs, and debugging are all shared.

## 15. Example Message Exchanges (Appendix A)

### Valid POST

POST 2 3 white Meeting next Wednesday

OK POST id=1

### Valid GET

GET color=white

OK NOTES count=1 notes=...

### Error Example

PIN 50 50

ERROR NO\_NOTE\_AT\_COORDINATE No note contains point 50 50



### **Concurrent Scenario**

Two clients POST concurrently; server assigns unique IDs and preserves consistency.

## **16. Revision History (Appendix B)**

| <b>Version</b> | <b>Date</b>    | <b>Description</b> |
|----------------|----------------|--------------------|
| 1.0            | 30 Jan<br>2026 | Initial version    |